

6. Testing and Evaluation

6.1 Testing Strategy

Repository Discovery Summary

Game Forge is a multi-service stack coordinated from the `game-forge/` orchestration root. The root owns Docker Compose, benchmark artifacts, benchmark scripts, and high-level documentation. The source services are separate sibling repositories:

Area	Technology	Primary Responsibility	Current Test Position
<code>game-forge/</code>	Docker Compose, Node package, Python benchmark scripts	Full-stack orchestration and benchmark artifacts	<code>npm test</code> only runs Python syntax compilation for benchmark scripts; benchmark scripts call live Groq APIs when executed directly.
<code>game-forge-frontend/</code>	React 18, Vite, Tailwind CSS	Browser UI, auth, project workspace, chat, build polling, game playback	No test runner or tests currently configured.
<code>game-forge-backend/</code>	Node.js, Express, Mongoose, MongoDB, MinIO	Public REST API, auth, canonical User/Chat/GameProject/GenerationJob/Build models, internal pipeline status endpoints	No test script or tests currently configured.
<code>game-forge-server/</code>	Node.js, Express, Undici/fetch	Stateless orchestrator that coordinates planner, asset, code, builder, and backend internal endpoints	Vitest and Supertest are installed, but no test files are present.
<code>game-forge-planner/</code>	Node.js, Express, Mongoose, Groq/OpenRouter	Prompt classification, plan generation, GamePlan persistence	No test script or tests currently configured.
<code>game-forge-asset/</code>	Node.js, Express, Mongoose, MCP client, Groq/OpenRouter, MinIO	Prompt-spec composition and deterministic MCP asset generation orchestration	No test script or tests currently configured.
<code>game-forge-code/</code>	Node.js, Express, Godot project assembly, MinIO	Archetype loading, GDScript assembly, behavior/weapon/hook validation, project upload	<code>package.json</code> references missing <code>test/run.js</code> and <code>test/features.test.js</code> ; no test directory exists.
<code>game-forge-builder/</code>	Node.js, Express, Godot CLI, MinIO	Downloads assembled project/assets, runs or skips Godot export, uploads HTML5 artifacts	One Node built-in test file exists for SFX selection.
<code>assets-mcp/</code>	Python 3.13, FastMCP, Pydantic, aiohttp, Pillow, MinIO	MCP tools wrapping Stability AI image/audio generation and object storage	No pytest configuration or tests currently configured.

The Docker Compose stack includes frontend, backend, server, planner, asset, code, builder, assets-mcp, MongoDB, and MinIO. The critical runtime path is:

- Frontend creates/refines a game project through backend `/api/*` .
- Backend persists project/chat/job/build records and starts generation through `game-forge-server` .
- Orchestrator calls planner first, then asset and code services, then builder.
- Worker services call LLM providers, MCP/Stability AI, MinIO, Godot CLI, and backend `/internal/*` endpoints.
- Backend exposes status and playable/downloadable game artifacts to the frontend.

Testing Goals

The testing foundation should make regressions visible without depending on paid APIs, production secrets, or heavyweight export tooling. The goals are:

- Protect request validation, service-auth behavior, model methods, and deterministic utilities.
- Verify the main HTTP contracts between frontend, backend, orchestrator, and worker services.
- Exercise generation orchestration using mocks for LLMs, Stability AI, MCP, MinIO, MongoDB where appropriate, Godot CLI, and inter-service HTTP calls.
- Produce coverage artifacts that CI can archive and compare over time, especially `coverage/lcov.info` for JavaScript services.
- Keep unit tests fast enough for local development and CI.
- Keep integration tests deterministic and isolated from production data and real external systems.

Testing Scope

The first testing pass will cover unit and integration tests only. It will not perform live LLM evaluation, live Stability AI generation, full Godot export benchmarking, browser end-to-end testing, load testing, or visual regression testing.

Unit scope includes pure functions, model methods that can be exercised without a database connection, request validation logic, middleware, service clients with mocked `fetch`, and frontend utilities/hooks/components with mocked browser APIs.

Integration scope includes Express HTTP route contracts through Supertest, service-auth checks, success and error envelopes, backend internal status endpoints, and orchestrator pipeline sequencing with mocked downstream services. Broader Docker Compose smoke tests are deferred to a practical subset after deterministic unit and service-level integration tests are passing.

Unit Testing Approach

Unit tests should be added before integration tests. The highest-value unit candidates are:

Service	Unit Test Candidates
Backend	<code>validateBody</code> , <code>requireServiceAuth</code> , <code>catchAsync</code> , <code>AppError</code> , JWT token helper with controlled env, <code>User.changedPasswordAfter</code> , <code>GenerationJob</code> methods, <code>Build</code> methods, controller validation paths with mocked models/services.
Server	<code>serviceClient.callService</code> , <code>backendClient</code> request construction and error handling, <code>generateController</code> validation and 202 behavior, <code>runPipeline</code> sequencing with mocked clients.
Planner	<code>extractJson</code> , <code>withRetry</code> , service auth middleware, plan route validation, plan generation fallback path with mocked provider and <code>GamePlan.create</code> .
Asset	<code>extractJson</code> , <code>withRetry</code> , service auth middleware, asset route validation, asset model virtual/default behavior, deterministic prompt/manifest helper behavior through exported or minimally extracted helpers.
Code	<code>extractJson</code> , <code>withRetry</code> , <code>fillSlots</code> , seeded RNG helpers, <code>validateHookSnippet</code> , <code>validateCustomBehaviorRegistry</code> , <code>injectHooks</code> , <code>composeEntity</code> , <code>composeWeapon</code> , <code>archetypeLoader</code> cache/list behavior, controller validation.
Builder	Existing SFX mapping, service auth middleware, build route validation, Godot service command handling with mocked <code>spawn</code> , builder success/error behavior with mocked storage/backend/Godot and <code>SKIP_GODOT_EXPORT</code> .
Frontend	<code>fetchApi</code> , <code>getAuthHeaders</code> , <code>api</code> request wrappers, <code>authStorage</code> , <code>workspaceUtils</code> , <code>ProtectedRoute</code> , <code>useAuthForm</code> , <code>useBuildPolling</code> , <code>useOptimisticChat</code> , compact component behavior for chat input/messages/sidebar status.
Python MCP	Pydantic input schemas, prompt helpers, audio format detection, storage object-key formatting with mocked MinIO, tool response envelopes, tool success/error paths with mocked Stability clients and storage.

Private helpers should not be exported only for convenience unless the extraction improves production code clarity. When extraction is necessary, it should be small, named by responsibility, and covered immediately.

Integration Testing Approach

Integration tests should start at the HTTP boundary and avoid real external calls. The first pass should cover:

- Health endpoints for every Express service.
- Service-auth enforcement for planner, asset, code, builder, backend `/internal`, and orchestrator production-secret behavior.
- Worker route validation and success/error envelopes:
 - `POST /plan`
 - `POST /assets`
 - `POST /code`

- `POST /build`
- Backend public API contracts:
 - auth happy/error paths,
 - protected route rejection,
 - project creation/list/read/delete with mocked or isolated persistence,
 - chat confirmation/status transitions with mocked LLM,
 - game status response shapes,
 - internal job/build/project status mutation endpoints.
- Orchestrator `POST /generate` acceptance and background pipeline dispatch.
- Orchestrator pipeline success and failure sequencing with mocked `backendClient` and mocked worker `callService`.
- Builder `runBuild` with storage and backend clients mocked, using skipped or mocked Godot export.

Full Docker Compose integration should be limited to smoke checks unless separately approved. The practical first Docker-level checks are service startup, `/health` endpoints, service-auth wiring, and a builder path using `SKIP_GODOT_EXPORT=true`. Live LLM, Stability AI, and full Godot export should not be part of the default integration suite.

Frontend Testing Approach

The frontend should use the Vite-compatible unit test stack: Vitest, React Testing Library, `@testing-library/user-event`, `@testing-library/jest-dom`, and `jsdom`. Tests should mock `fetch`, `localStorage`, routing, and the auth/workspace contexts where appropriate.

Frontend tests should prioritize behavior users rely on:

- Authenticated and unauthenticated route behavior.
- Auth form validation and mode switching.
- API wrapper headers, body serialization, unauthorized event dispatch, and error handling.
- Workspace URL conversion and empty build state creation.
- Chat input send/confirm/build button enablement.
- Build polling state updates, retry cutoff, ready-state handling, and cleanup.
- Optimistic chat append/rollback behavior.

Visual style and animation quality should not be asserted in unit tests. Tests should verify accessible labels, button states, rendered text, and callback behavior.

Backend and Microservice Testing Approach

JavaScript services should converge on Vitest for unit tests, Supertest for Express integration tests, and `@vitest/coverage-v8` for LCOV coverage. `game-forge-server` already uses Vitest and Supertest, so this aligns the rest of the Node services with an existing local convention. `game-forge-builder` currently uses Node's built-in test runner for one file; moving that suite to Vitest is justified by the need for consistent mocking and coverage reporting.

Backend tests should isolate MongoDB-dependent behavior with either mocked Mongoose models or a dedicated in-memory/test MongoDB strategy. Unit tests should prefer mocks; integration tests may use `mongodb-memory-server` for backend persistence-heavy flows if the environment supports it. If binary download or CI reliability becomes a problem, the fallback is mocked-model route integration plus a Docker Mongo smoke suite.

Worker service tests should not require real MongoDB, MinIO, LLM keys, MCP server availability, or Godot. Route-level integration should mock the service implementation underneath the route. Service-level unit tests should mock provider factories, MCP clients, MinIO clients, backend clients, and process spawning.

Python MCP Service Testing Approach

`assets-mcp/` should add a Python test configuration using `pytest`, `pytest-asyncio`, and `pytest-cov`. The first tests should cover:

- Pydantic schema validation for sprite/background/music inputs.
- `build_image_prompt`, `build_negative_prompt`, `prompt_slug`, and `build_audio_prompt`.
- `detect_audio_format`.
- MinIO object-key and URL formatting with a mocked MinIO client.
- Tool success paths with mocked Stability clients and mocked storage.
- Tool error envelopes when providers or storage fail.
- MCP server tool wrappers by calling wrapper functions directly, not by starting a live MCP transport server.

No Python test should call Stability AI or a real MinIO service by default.

Mocking and Isolation Strategy

External systems must be isolated as follows:

Dependency	Isolation Strategy
Groq and OpenRouter LLM providers	Mock provider factories or provider classes. Assert request shape and fallback behavior, never call live APIs.
Stability AI image/audio APIs	Mock <code>stability_image_client</code> and <code>stable_audio_client</code> ; verify prompts and response handling only.
MongoDB	Use mocked Mongoose models for unit tests; use <code>mongodb-memory-server</code> only for selected backend integration tests if reliable in the environment; otherwise use Docker Mongo smoke tests outside the default suite.
MinIO	Mock storage service modules and MinIO clients in unit/integration tests; reserve real MinIO for optional Docker smoke tests.
Godot CLI	Mock <code>child_process.spawn</code> and <code>godotService</code> ; use <code>SKIP_GODOT_EXPORT=true</code> or dummy outputs for builder service integration.
MCP HTTP calls	Mock MCP client connection and <code>callTool</code> ; do not start the Python MCP service for Node unit tests.
HTTP calls between services	Mock <code>fetch</code> , <code>callService</code> , and <code>backendClient</code> for unit tests; use Supertest for in-process Express route tests.
Browser APIs	Mock <code>fetch</code> , <code>localStorage</code> , <code>window.URL</code> , <code>window.confirm</code> , timers, and router context in jsdom tests.

Test suites must not require production `.env` files or production secrets. Tests should set minimal environment variables inside setup files or per-test scopes.

Test Data Strategy

Test data should be small, explicit, and local to each suite:

- Use minimal valid plan fixtures for platformer, top-down, and endless-runner cases.
- Use deterministic IDs such as `project-1` , `game-1` , `job-1` , and `build-1` unless a real `ObjectId` is required.
- Use small binary buffers for image/audio/storage tests.
- Use seeded RNG values when testing procedural helpers.
- Avoid checking in generated coverage, build artifacts, or large generated assets.
- Keep fixtures close to their tests unless they are reused by multiple services.

Coverage Expectations

The first pass should establish coverage generation, not chase arbitrary thresholds. Recommended initial expectations:

Area	Initial Coverage Expectation
Pure utilities and validators	High coverage, targeting important branches and error cases.
Controllers and route validation	Cover required-field errors, auth rejection, success envelope, and service errors.
Orchestrator sequencing	Cover success, missing downstream data, downstream failure, and backend failure marking.
Frontend utilities/hooks/components	Cover core states and user-visible behavior.
Python MCP utilities/tools	Cover prompt, schema, storage formatting, success envelope, and error envelope.
Full generation/export	Smoke coverage only with mocks or skip flags in this pass.

After the baseline is stable, coverage thresholds can be introduced per repository. Starting with strict thresholds before the foundation exists would create noise rather than useful signal.

CI Compatibility

JavaScript coverage should be emitted as `coverage/lcov.info` in each service repository. Vitest with V8 coverage can produce that artifact for each JavaScript service, and CI can archive those reports or feed them into future quality tooling.

Recommended JavaScript scripts per service:

Script	Purpose
<code>test</code>	Run all unit and integration tests once.
<code>test:unit</code>	Run unit tests only.
<code>test:integration</code>	Run service-level integration tests only.
<code>test:coverage</code>	Run tests and write LCOV coverage under <code>coverage/lcov.info</code> .

The Python MCP service should emit terminal and XML coverage initially. This keeps the default test workflow independent of any hosted analysis service while still producing machine-readable coverage.

What Will Not Be Tested Automatically in This Pass

The following areas are intentionally outside the first automated test pass:

- Live LLM answer quality from Groq or OpenRouter, because it is nondeterministic, paid/quota-bound, and already represented by separate benchmark artifacts.
- Live Stability AI image and audio generation, because it is paid/quota-bound and output quality is not deterministic.
- Full browser end-to-end flows, because the first need is a unit and service integration foundation.
- Full Godot HTML5 export by default, because it is slow, environment-sensitive, and depends on a correctly installed Godot CLI.
- Visual regression for generated games and frontend styling, because screenshot baselines are not yet established.
- Load, soak, and concurrency testing, because they require a stable functional baseline first.

Risks and Mitigations

Risk	Mitigation
Tests accidentally call live paid APIs.	Mock provider factories by default, fail fast when tests see real API keys are required, and avoid importing live clients in route tests where possible.
MongoDB integration tests are flaky because in-memory Mongo downloads binaries.	Keep backend unit tests model-mocked; use <code>mongodb-memory-server</code> only for selected flows; fall back to Docker Mongo smoke tests if needed.
Godot export tests are slow or unavailable in CI.	Default to <code>SKIP_GODOT_EXPORT=true</code> or mocked <code>godotService</code> ; keep full export as a manually approved smoke/benchmark path.
MinIO tests become environment-dependent.	Mock storage service modules in default suites; reserve real MinIO for optional Docker Compose smoke checks.
Cross-repository scripts diverge.	Standardize JavaScript services on Vitest/Supertest/LCOV scripts and keep frontend on the same Vitest coverage tooling.
Existing package scripts are missing or broken.	Update scripts as part of the test tooling phase, especially <code>game-forge-code</code> where <code>test/run.js</code> is referenced but absent.
Test setup causes lint failures.	Add test globals/config patterns when test files are introduced, and run lint for each affected repository.

6.2 Unit and Integration Tests, and Their Results

Test execution is pending. No unit or integration tests have been implemented or run as part of the strategy phase. Results will be filled after the approved implementation phase, using actual command output only.

Area	Test Type	Command	Result	Coverage / No
game-forge-backend	Unit	npm run test:unit	Pending execution	Will cover middle utilities, model methods, and controller validatic
game-forge-backend	Integration	npm run test:integration	Pending execution	Will cover Express route contracts wi mocked or isolate persistence and external clients.
game-forge-server	Unit	npm run test:unit	Pending execution	Will cover service clients, controller validation, and pip sequencing.
game-forge-server	Integration	npm run test:integration	Pending execution	Will cover /health auth behavior, and /generate acceptance.
game-forge-planner	Unit	npm run test:unit	Pending execution	Will cover JSON extraction, retry behavior, auth middleware, and planner fallback behavior.
game-forge-planner	Integration	npm run test:integration	Pending execution	Will cover /health service auth, and /plan route envelopes.
game-forge-asset	Unit	npm run test:unit	Pending execution	Will cover JSON/r utilities, auth middleware, asse model behavior, a mocked asset orchestration.
game-forge-asset	Integration	npm run test:integration	Pending execution	Will cover /health service auth, and /assets route envelopes.
game-forge-code	Unit	npm run test:unit	Pending execution	Will cover determ pipeline utilities at validators.
game-forge-code	Integration	npm run test:integration	Pending execution	Will cover /health service auth, and /code route envelopes.
game-forge-builder	Unit	npm run test:unit	Pending execution	Will include existin SFX coverage and builder helper beh
game-forge-builder	Integration	npm run test:integration	Pending execution	Will cover /health service auth, and /build with

Area	Test Type	Command	Result	Coverage / No
				Godot/storage/ba mocked or skippe
game-forge-frontend	Unit/component	npm run test:unit	Pending execution	Will cover utilities, hooks, route guar API wrappers, and selected compone
assets-mcp	Unit	python -m pytest	Pending execution	Will cover schemat prompt/audio help storage formatting mocked tool envelopes.
JavaScript services	Coverage	npm run test:coverage per service	Pending execution	Expected to gene coverage/lcov.in for CI artifacts.
Python MCP service	Coverage	python -m pytest --cov=app --cov-report=term-missing --cov-report=xml	Pending execution	Expected to gene terminal coverage coverage.xml .

6.3 Performance Evaluation / Benchmarking

Performance benchmarking is outside the scope of this unit and integration testing pass. Existing benchmark artifacts are already present under:

Benchmark Area	Existing Artifact Directory
Planner LLM behavior	game-forge/docs/planner benchmark/
Code/coder model behavior	game-forge/docs/coder benchmark/
Theme prompt behavior	game-forge/docs/theme benchmark/
Audio prompt behavior	game-forge/docs/audio benchmark/

The existing scripts in `game-forge/scripts/` are benchmark/evaluation scripts that call live Groq APIs when executed directly. They should remain separate from the deterministic automated test suite.

Future benchmarking work can add reproducible timing for service startup, mocked pipeline latency, Docker Compose smoke latency, and optional full Godot export duration. Those measurements should be reported separately from unit and integration test pass/fail results.

6.4 Limitations and Known Issues

Limitation / Issue	Impact	Current Position
Live LLM providers are mocked in automated tests.	Tests validate request handling and fallback logic, not real model quality.	Model quality remains covered by benchmark artifacts and manual evaluation.
Stability AI is mocked in automated tests.	Tests validate tool orchestration and response handling, not generated media quality.	Live generation requires explicit approval and valid quota/secrets.
Godot export is skipped or mocked by default.	Tests cannot prove every generated project exports successfully.	Full export is environment-sensitive and should be a separate approved smoke/benchmark path.
MongoDB integration strategy may vary by environment.	<code>mongodb-memory-server</code> can be slower or download-dependent.	Use mocked models first; use in-memory Mongo selectively; fall back to Docker Mongo smoke checks if needed.
MinIO is mocked in default suites.	Object storage credentials and network behavior are not exercised by default.	Real MinIO checks belong in Docker Compose smoke tests.

Limitation / Issue	Impact	Current Position
No frontend browser end-to-end suite in the first pass.	Cross-browser behavior and complete UI journeys are not automatically verified.	Unit/component tests should stabilize behavior first; Playwright can be added later.
<code>game-forge-code</code> has broken test script references.	Current <code>npm test</code> cannot be considered a working test suite.	Scripts should be corrected during test tooling setup.
Python MCP has no current dev/test dependency split.	Installing test tools through production requirements would bloat the runtime image.	Add a dev requirements file for pytest tooling.
Separate service repositories do not share a root workspace.	One command cannot currently run all service tests cleanly without orchestration scripts.	Add per-service scripts first; add root aggregation only after the suites are stable.